

INDEPENDENT TECHNOLOGY BY HERITEON

All-in-one. Built around data.



GravitEon

Sorting · Lookup · Grouped data

An engine in development, bringing ordering, fast lookup and grouped financial data into one software foundation.

97/97

test cases passed

10M

largest input size

95.5%

lookup comparisons led

This first technical preview shows the measured gains, the trade-offs and the next tests.

01 / THE SCOPE

One engine. Three kinds of work.

01 Sort

Serial and parallel ordering of byte and UTF-16 keys.

Compared with C++ `std::sort` and Microsoft PPL sorting algorithms.

02 Lookup

Index construction, successful searches and several forms of missing-key queries.

Compared with `ankerl::unordered_dense` 4.5.0, Abseil `flat_hash_map` 20250127.1, `tsl::robin_map` 1.3.0 and Boost `unordered_flat_map` 1.88.0.

03 Grouped financial data

Snapshot construction, validation and queries over grouped records.

Compared with the benchmark's Standard C++ Bank Pipeline reference implementation.

“All-in-one” describes the scope of GravitEon. These comparisons test individual operations; they do not rank complete database or banking platforms.

Reference implementations: [ankerl](#) · [Abseil](#) · [Tessil](#) · [Boost](#)

02 / THE COMPARISON FIELD

Who the engine is measured against.

The rivals are established C++ building blocks: libraries chosen specifically for sorting or fast key lookup.

Abseil / flat_hash_map

A Swiss-table hash container from Google's Abseil C++ library. Its flat storage is designed for efficient lookup and memory use.

Boost / unordered_flat_map

Boost's open-addressing hash map. A performance-oriented alternative to a traditional node-based unordered map.

ankerl / unordered_dense

A specialist hash map with densely stored values. It is the fastest rival in several of the selected lookup examples in this preview.

Tessil / robin_map

A C++ hash map using Robin Hood hashing. It wins some of the large tiny-key lookup cases in this run.

Microsoft PPL + the C++ standard library

Parallel sorting algorithms and `std::sort` provide the sorting baselines. PPL Buffered Sort wins selected mid-size UTF-16 cases.

These are comparisons with specific library versions and APIs, not claims of outperforming the companies behind them. The Bank suite uses a separate in-harness C++ reference.

03 / THE COMPLETE MEASURED SET

How often does GravitEon lead?

Every result is counted, including ties and losses. The groups have different sizes and are shown separately.

Lookup queries 95.5% lead



548 leads · 10 close · 16 trails / 574 comparisons

Lookup construction 92.7% lead



76 leads · 2 close · 4 trails / 82 comparisons

Serial sorting 93.9% lead



77 leads · 2 close · 3 trails / 82 comparisons

Parallel sorting 91.5% lead



75 leads · 2 close · 5 trails / 82 comparisons

Bank snapshot + validation 100.0% lead



15 leads · 0 close · 0 trails / 15 comparisons

Bank group queries 100.0% lead



15 leads · 0 close · 0 trails / 15 comparisons

■ Lead ■ Within ±3% ■ Trail

Percentages count comparisons, not percentage speed gains. Each comparison uses the fastest eligible rival median. Leads/trails require more than a 3% time difference; this is a display band, not a confidence interval.

04 / SORTING

Ten million keys. Two execution modes.

Serial

5.51×

rival time / GravitEon time

GravitEon Sort

857.75 ms

std::sort

4,725.00 ms

Parallel · 4-worker limit

3.67×

rival time / GravitEon time

GravitEon Parallel Sort

301.83 ms

Microsoft PPL Buffered Sort

1,106.40 ms

Large random-key inputs are a strong result in this run. Small inputs and some mid-size UTF-16 cases are less favourable.

Input: random_short, byte keys, n = 10,000,000, seed 731. Median elapsed time; lower is better. Parallel comparison uses the fastest eligible rival under the 4-worker limit.

05 / THE LARGEST MEASURED GAINS

Beyond 10x. On specific queries.

15.11x

93.4% less query time

1M same_edges keys · UTF-16
Appended-key misses

GravitEon Lookup

1.386 ns/query

ankerl::unordered_dense

20.951 ns/query

11.65x

91.4% less query time

10M hard_prefix keys · UTF-16
Early-key misses

GravitEon Lookup

3.044 ns/query

ankerl::unordered_dense

35.468 ns/query

Both ratios use the fastest rival for that test. Nine of 574 lookup-query comparisons exceed 10x. These are selected peaks, not an average or an end-to-end application speedup. Successful hits on the same_edges UTF-16 dataset are slower; see page 10.

06 / LOOKUP

The query matters.

548 leads across 574 query comparisons. The strongest consistency is in hot hits and two missing-key patterns.

Query pattern · 82 cases each	Leads	Close	Trails
Successful lookup	73	3	6
Repeated hot-key hits	82	0	0
Appended-key misses	82	0	0
Early-key misses	82	0	0
Mutated-key misses	77	3	2
Same-alphabet misses	78	3	1
Mixed hits and misses	74	1	7

10M RANDOM_SHORT · UTF-16 · HITS

43.59 ns/query

GravitEon Lookup

56.89 ns/query

ankerl::unordered_dense

Fastest rival in this case

Other patterns expose trade-offs. UTF-16 keys with shared beginnings and endings are the clearest weakness; exact results are shown on page 10. Query medians use the supplied 8,192-query workloads.

07 / FINANCIAL DATA

Group by the dimensions the work requires.

Legal entity, portfolio, product, country, category, date, scenario and identifier.

RECORDS → VALIDATED SNAPSHOT → GROUP QUERY

5.64x

group-query time ratio

1.29x

snapshot-build time ratio

10M regulatory_dims records	GravitEon	C++ reference
Group query	201.33 ns	1,134.95 ns
Build, copy and validate	30.33 s	39.20 s

Basel-oriented data preparation

The scenario dimension and exact aggregate checks provide a basis for exploring regulatory reporting workloads.

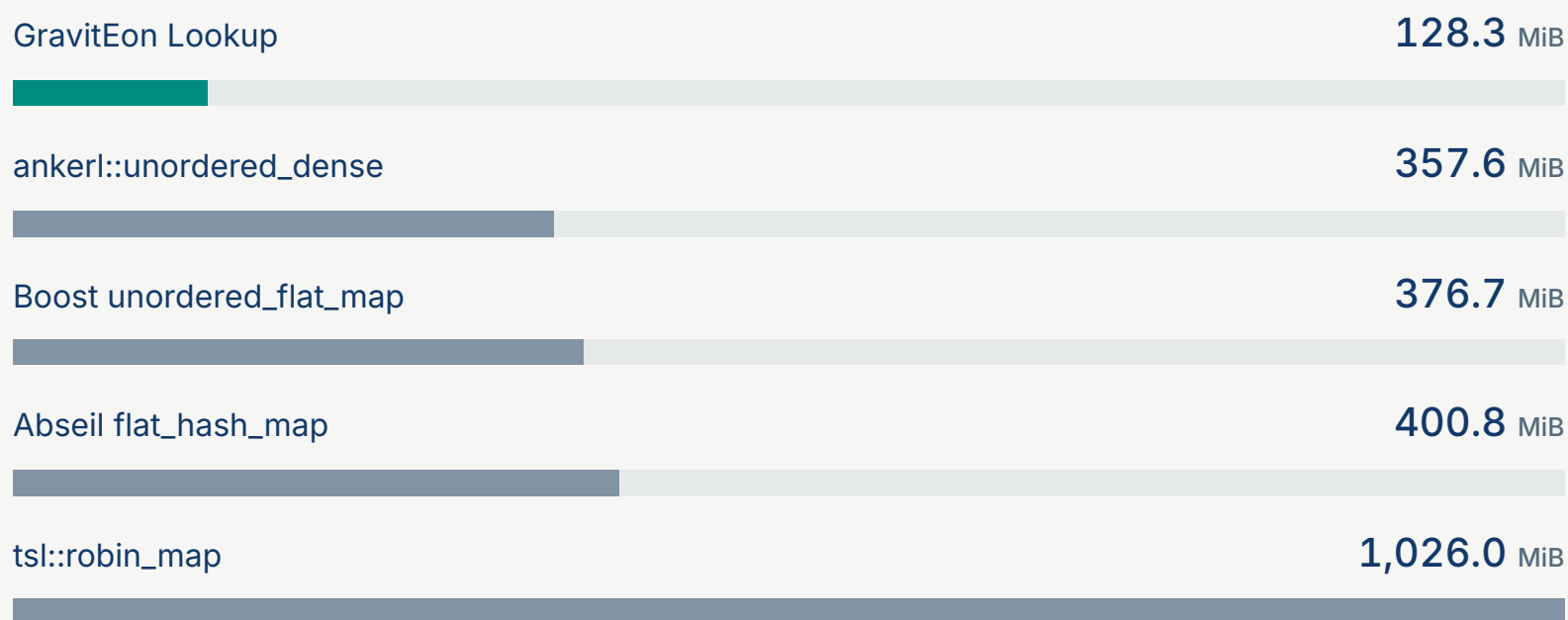
Scope: synthetic data preparation relevant to Basel II / final Basel III workflows, including UK Basel 3.1. Capital calculations, RWA rules and regulatory compliance are outside this benchmark.

Framework scope: [Basel Committee, final Basel III reforms](#) · [PRA, Basel 3.1](#)

08 / RESOURCE USE

Memory matters, too.

A separate resource pass measures index construction on 10 million random_short UTF-16 keys.



Index construction	Extra peak MiB	CPU time, ms
GravitEon Lookup	128.25	593.75
ankerl::unordered_dense	357.59	843.75
Boost unordered_flat_map	376.74	1,125.00
Abseil flat_hash_map	400.79	1,328.12
tsl::robin_map	1,026.01	1,359.38

Extra peak = sampled process-private memory above the prepared-input baseline, not total process RAM or live allocations. Sampling interval: 250 ms; brief peaks may be missed. CPU time is accumulated across threads. These figures describe construction, not query CPU cost.

09 / WHERE IT TRAILS

The difficult cases stay in the report.

LOOKUP / 1M SAME_EDGES / UTF-16

Shared beginnings and endings

Successful lookups take **251.32 ns** in GravitEon versus **105.85 ns** in `ankerl::unordered_dense`. GravitEon takes 2.37× as long.

The same dataset also trails on mutated-key misses and mixed queries.

PARALLEL SORT / 32,767 RANDOM_SHORT KEYS / UTF-16

Mid-size inputs

GravitEon takes **1.8191 ms**; Microsoft PPL Buffered Sort takes **1.0898 ms**. Parallel overhead deserves attention at this scale.

SMALL INPUTS AND SELECTED LOOKUP PATTERNS

No blanket speed claim

There are losses on tiny sorts, some 128-key index builds and selected large-input hit or mixed-query workloads.

Very short sort timings are limited by the precision recorded in the source CSV. All leads and losses are retained in the accompanying comparison table.

10 / NEXT VALIDATION

Public datasets next.

The current results use controlled synthetic workloads. The next phase will test externally sourced data and publish the same timing and correctness checks.

01 PaySim

Public synthetic mobile-money transactions, including injected fraudulent behaviour. Planned checks: account lookup, sorting, grouped totals and data conversion.

[Kaggle dataset](#) · [Original simulator](#)

02 Financial scenario research

The CFA Institute RPC repository is a reference for selecting synthetic-finance methods and case studies. It is not a ready-made bank stress-test dataset.

[CFA Institute RPC / Synthetic Data for Finance](#)

03 Controlled variations

SDV is a candidate for tabular and relational synthesis. Faker can supply formatted identifiers such as IBANs and SWIFT codes; it does not model financial behaviour.

[SDV documentation](#) · [Faker bank provider](#)

Planned work only: no PaySim, CFA, SDV or Faker results are included in this preview. Record dataset version, file hash, transformations and usage terms before each run. Fraud labels do not make GravitEon a fraud-detection model.

11 / TEST RECORD

Measured on a disclosed platform.

AMD Ryzen 9 3900X

12 cores / 24 logical processors

MSI B550 Tomahawk

64 GB Kingston FURY Beast

4 × 16 GB · single-rank · DDR4-3600 CL18

CPU identified in the run; motherboard and memory specification supplied by the developer.

Build

Windows x64

MSVC 19.38.33145

Assertions disabled

Parallel limits

Up to 4 workers.

Rivals must not exceed the GravitEon worker limit. AUTO runs excluded.

Timing

3 repeats on large cases

5 on small cases

Medians, not best runs

Checks

97/97 cases passed.

Matched input/query hashes.

932 resource operations recorded separately.

Source run

run_20260925_015224

Executable SHA-256

9687878ba29d8798e48a9c9727a6a6a66
b6d6357a71bc1e04efcea73b0d243ad

Developer measurements on one machine, not independent certification. Results apply to these inputs, library versions, hashers and build settings. Other machines and workloads may differ.

heriteon.com

[GitHub / HeritEon-Dev](#)

office@heriteon.com